Budget-Constrained Reinforcement Learning for Human-in-the-Loop Agent Orchestration

Project report: problem, design, implementation, every result, what we can and cannot claim

Document	Full project history, start to finish (October 2026)
Data	HotpotQA (distractor setting), real questions, real gold answers and gold supporting paragraphs. No synthetic data anywhere in the final system.
Models	Qwen2.5-3B-Instruct (base agents), Qwen2.5-7B-Instruct-AWQ (specialist), bge-small-en-v1.5 (embeddings), DeBERTa-v3 NLI (verifier)
Compute	Kaggle (T4 GPU for the real pipeline runs via vLLM; CPU for policy training and evaluation)
Final code version	1.2-confirm (earlier versions: 0.6-ppo-decode, 0.7-comparisons-csv, 1.0-final, 1.1-stacked)
Status	Main study complete. Confirmatory evaluation on fresh queries is prepared and the fresh cache is built; the confirmatory evaluation itself has NOT been completed yet (Section 10).

Executive summary

Problem. Enterprise multi-agent workflows either run fully autonomously (risking confident hallucinations) or ask humans to review too many steps. Fixed confidence thresholds fail because LLM self-reported confidence is poorly calibrated. The goal was an orchestration agent that decides, step by step, whether to advance automatically, re-retrieve with modified constraints, route to a stronger specialist agent, or interrupt for human review, while strictly staying inside a human-review budget per 1,000 queries, and that leaves an audit trail.

Approach. We replaced simulated environments with a real pipeline (retrieval, summarisation, answering, critique, NLI verification) executed on HotpotQA. Every action the orchestrator could take was executed once and cached (cached-counterfactual replay), so policies train and evaluate on real outputs with real ground truth. We formulated the decision problem first as a per-query constrained MDP (PPO-Lagrangian), then as the production setting: a **hard cap of K human reviews per window of 1,000 queries** enforced by action masking, with the policy observing its remaining budget. A final variant gives the policy an out-of-fold supervised risk score as an input feature ("risk-stacked").

Key findings.

- Most of the benefit comes from the automatic pipeline upgrades and from spending human reviews on the highest-risk answers: wide re-retrieval alone adds +0.080 net reward over the cheap default, the 7B specialist adds +0.019 alone (+0.083 together), and human reviews add +0.016 (b=0.02) to +0.195 (b=0.30) on top.
- A plain PPO policy does **not** clearly beat a strong non-RL baseline (always re-retrieve, supervised risk model, budget pacing) given the same actions and the same hard cap: 0 wins, 12 ties, 3 losses in 15 comparisons.
- Giving the policy the supervised risk score improved it in every one of 15 cells (+0.005 to +0.021 net reward): 4 wins, 10 ties, 1 loss. All 12 cells with budget up to 0.20 had a positive point estimate (+0.001 to +0.017); the 3 cells at b=0.30 were negative (-0.004 to -0.014).
- Error-prediction signals are the real bottleneck: the best single signal reaches about 0.68 AUROC, the combined supervised model about 0.77, while the hindsight oracle is 0.15 to 0.23 net reward above the learned policy.
- Budget compliance was perfect in all stream evaluations (0 windows over budget).

Status and honest reading. The stacked-policy gains are small (about 0.5 to 1.7 points of net reward), individually borderline (lower confidence bounds +0.0007 to +0.0042), and were found after design choices were guided by test-set numbers. A pre-registered confirmatory evaluation on 2,000 fresh queries is therefore prepared; its cache is built, but the evaluation has not been run to completion. Until it is, the correct claim is "consistent small improvement, not yet independently confirmed", not "RL beats heuristics".

1. Problem and motivation

Modern enterprise workflows chain several LLM agents (retrieval, summarisation, critique, verification) to produce technical or legal syntheses. Two failure modes dominate:

- **Full autonomy** lets a wrong intermediate step propagate into a confident, wrong final output.
- **Reviewing too many steps** overwhelms human reviewers and defeats the purpose of automation. Human attention is a hard, scarce resource: the realistic constraint is "at most K reviews per 1,000 queries".

A common fix is a fixed confidence threshold on the agent's self-reported confidence. This fails because LLM verbalised confidence is badly calibrated and the right threshold depends on the budget and on how difficult the current stretch of queries is. The research question therefore became: *can a learned orchestration policy, trained on real pipeline behaviour, allocate a limited human-review budget (and cheaper automatic escalations) better than strong non-learned rules?*

The intended contribution was: a reliable, auditable orchestrator that bridges experimental agent frameworks and production constraints (reliability, cost, human engineering time).

2. Starting point and the decision to redesign

The project began from an earlier framework (RLHIL: a budget-constrained CMDP for human-in-the-loop oversight, written up in the uploaded paper). That work was evaluated on simulated environments and reported very large effect sizes against trivial baselines such as "always intervene". For this project the requirement was stricter: **no synthetic data**. This forced several design consequences:

Consequence of "no synthetic data"	How we handled it
Uncertainty must be measured, not simulated	Run real LLM agents; read token log-probabilities, critic Yes/No probabilities, NLI scores, retrieval scores, sampled-answer agreement.
Correctness needs ground truth	Use HotpotQA (distractor): gold answers and gold supporting paragraphs give truth for the answer and for the retrieval step.
Actions not taken have unknown outcomes	Cached-counterfactual replay: execute every branch once (3 evidence variants x 2 answerers = 6 cells per query) and train/evaluate by replay.
Humans cannot be simulated with invented behaviour	The reviewer is an oracle that supplies the gold answer/evidence; this is the single explicit modelling assumption (sensitivity switch: human accuracy).
Easy baselines would inflate results	Compare against strong, matched-action-set heuristics, not only against "never/always intervene".

3. System design

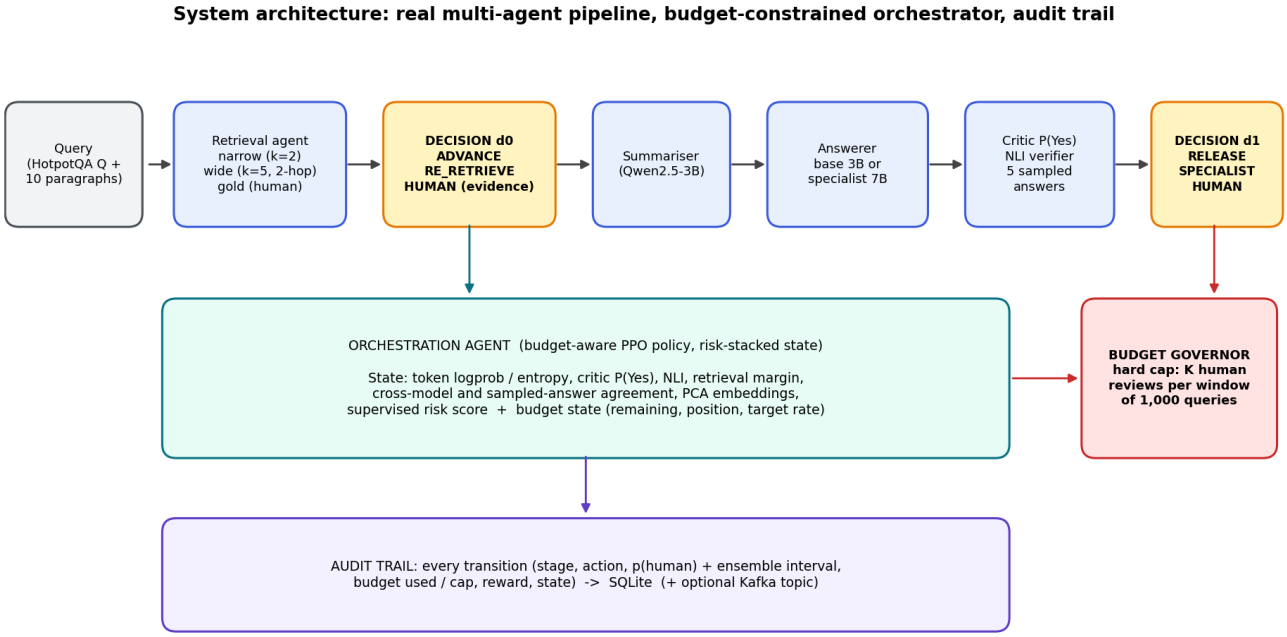


Figure 1. Architecture. Two decision points (after retrieval, after answer verification); a hard budget governor; every transition logged.

3.1 Data and splits

HotpotQA distractor: each question comes with 10 paragraphs (2 gold, 8 distractors), a gold answer and gold supporting facts. We shuffled with a fixed seed (42) and used 3,000 train, 1,000 validation (both from the official train split) and 2,000 test queries from the official dev split. The test set was used for final comparisons; validation was used for checkpoint selection and threshold calibration. The confirmatory study uses 2,000 further, disjoint dev queries (shuffled positions 2000 to 4000).

3.2 Agents and models

Agent	Implementation	Output used by the orchestrator
Retrieval	bge-small-en-v1.5 dense scores. **narrow**: top-2. **wide** (the "re-retrieve with modified constraints" action): top-5 with a second-hop query expanded by the top paragraph. **gold**: what a human would supply.	Top-1 score, margin, softmax entropy over the 10 paragraphs, mean/min selected score, k, mean paragraph embedding
Summariser	Qwen2.5-3B-Instruct, at most 5 factual bullets	Notes passed to the answerer
Answerer	Base: Qwen2.5-3B-Instruct. Specialist: Qwen2.5-7B-Instruct-AWQ (the "route to specialist" action). Greedy decoding plus 5 sampled answers (T=0.7)	Answer, token log-prob / entropy statistics, sampled-answer agreement
Critic	Qwen2.5-3B-Instruct, one-word Yes/No verdict; confidence read from next-token probabilities (not self-reported text)	P(Yes)
Verifier	DeBERTa-v3 NLI (premise: evidence, hypothesis: question + answer)	P(entailment), P(contradiction)

3.3 Measured uncertainty signals (the orchestrator's state)

Signal	Meaning
mean/min token log-prob, mean/max token entropy, answer length	Answerer's own token-level uncertainty (top-20 entropy approximation)
critic P(Yes), NLI entailment/contradiction	Independent judgement that the answer is supported by the evidence
xgap, xent	Critic-vs-NLI disagreement and entropy (cross-agent)
sem_agree, sem_ent	Mean F1 of 5 sampled answers vs the greedy answer; entropy over clusters of sampled answers (semantic-entropy style)
xmodel_agree	F1 between the base and the specialist answer; observable only after the specialist has been paid for
PCA embeddings (8-d each)	Question, retrieved evidence, answer
supervised risk score (stacked variant)	Out-of-fold GBM probability that the answer is wrong, per evidence/answerer cell
budget state (stream variants)	Remaining budget fraction, position in the window, pace (remaining minus remaining queries), target rate

3.4 Decision process, actions, reward

- **Stage R** (after retrieval): ADVANCE, RE_RETRIEVE (narrow to wide; once), HUMAN (human supplies gold evidence; the pipeline then continues).
- **Stage A** (after summarise, answer, critique, verify): RELEASE, SPECIALIST (re-run with the 7B answerer; once), HUMAN (human supplies the gold answer; episode ends). After a specialist re-run there is one more release/human decision.
- **Reward** = answer F1 minus compute costs (re-retrieve 0.02, specialist 0.05; both are our assumptions and were swept). A human review yields F1 = 1.0 (oracle).
- **Constraint**: each HUMAN action costs 1 review. Per-query form: expected reviews per query at most b. Stream form: at most K = round(b x W) reviews per window of W queries, enforced by masking the HUMAN action once the budget is exhausted.

3.5 Cached-counterfactual replay

For every query we computed all six cells (narrow/wide/gold evidence x base/specialist answerer) with all signals. The environment then simply looks up the outcome of whichever path the policy chooses. This keeps everything real (real model outputs, real uncertainty, real correctness) while making RL training take minutes instead of GPU-days. One caveat: re-running the GPU stages gives slightly different cells (batched greedy decoding in vLLM is not bit-reproducible; observed differences of about 0.002 to 0.005 in net reward), so one cache file is treated as the dataset.

3.6 Audit trail

Every transition is written to a SQLite table: timestamp, run id, query id, step, stage, action, the policy's mean probability of choosing HUMAN with a min/max interval across ensemble members, budget used and cap, reward, cost, and the full state vector. An optional Kafka producer mirrors rows to a topic. Live streaming to Kafka was implemented but not exercised in these experiments.

4. Learning formulations

4.1 Per-query constrained MDP: PPO-Lagrangian

Maximise expected net reward subject to expected human reviews per query at most b. Implemented in plain PyTorch (no Ray/RLlib, which also avoided a Windows DLL problem on the local machine): masked categorical actor, two critics (reward, cost), advantage $A_r - \lambda A_c$, dual ascent on λ using the batch's mean cost. One policy per budget (0.02, 0.05, 0.10, 0.20, 0.30), 5 seeds, ensemble by mean action probability. Validation-based checkpoint selection (net reward penalised if the review rate exceeds the budget) was added after the first small run showed overfitting.

4.2 Stream-level hard budget (the topic as stated)

"Per 1,000 queries" is a window constraint, not a per-query average. We therefore built a budget-augmented MDP: a window of W queries is processed sequentially; the state gains four features (remaining budget fraction, position in window, pace, target rate); the HUMAN action is masked when no budget remains, so the constraint holds by construction and no Lagrange multiplier is needed. One policy is trained for all budgets: training budgets are sampled per window in [0.02, 0.30] (log-uniform in version 1.0; half log-uniform, half uniform from 1.1, for better coverage of high budgets); training windows have W=200 (features are fractions, so the policy transfers to W=1,000); half of the training windows are re-ordered into homogeneous blocks by real metadata (question type or difficulty level) to expose the policy to drift. PPO with GAE (λ=0.98), 200 iterations in version 1.0 and 250 in 1.1, 3 seeds trained in parallel, validation-based checkpoint selection.

4.3 Risk-stacked state

The plain stream policy had to learn risk *ranking* from sparse rewards, which is exactly what a supervised model does well. In the stacked variant a gradient-boosted classifier (one per evidence/answerer cell) predicts P(answer wrong) from the same signals, and its logit (z-scored) is appended to the state. To avoid leakage, training queries receive 5-fold **out-of-fold** predictions; validation/test/fresh queries receive the model fit on all training queries, exactly as at deployment. The policy therefore only needs to learn pacing and escalation choices on top of calibrated risk.

4.4 Decoding the policy

A budget-feasible stochastic policy usually has p(HUMAN) below 0.5 everywhere, so argmax decoding never chooses a human at low budgets (we observed a human rate of exactly 0.000 at b=0.02 and 0.05 in the first evaluation). We therefore evaluate by **sampling from the (ensemble-mean) action distribution**, as trained, and fixed this as the primary decoding before the main and confirmatory runs; argmax and a calibrated p(HUMAN)-threshold decoding were kept as diagnostics.

5. Baselines and evaluation protocol

Baseline	Description
auto_only	Cheap default (narrow evidence, base answerer), no humans
always_specialist, wide_only, wide_spec_no_human	Fixed automatic upgrades, no humans
random_human, conf_logprob, critic_pyes, conf_semantic	Spend the budget on random queries or on the lowest single-signal confidence (threshold calibrated on validation to the budget)
risk_gbm	Supervised risk model + budget-quantile threshold on the cheap default path (human-only action set)
wide_gbm, wide_spec_gbm	Same action set as the policy: always re-retrieve (and always call the specialist), then spend humans on the highest GBM risk
static / paced variants (stream)	Static: fixed validation-calibrated threshold. Paced: threshold re-computed from remaining budget / remaining queries (online budget pacing)
oracle_hindsight	Unachievable ceiling: best automatic path per query; humans spent on the largest hindsight gains

Metrics. Net reward (F1 minus compute cost; human review counts as F1 = 1), F1, human/specialist/re-retrieve rates, budget violations. **Statistics.** Paired bootstrap over test queries (per-query values averaged over stream orderings), 95% CIs; verdict WIN/LOSS only if the CI excludes zero, otherwise TIE. **Streams** (all real queries, only re-ordered): iid; type-blocks (blocks of 100 same-type questions); hard-first (ordered by ground-truth difficulty; a stress test, not a deployment estimate). **Hard cap:** identical for every method (HUMAN masked when exhausted). Evaluation windows W=1,000 (2 per ordering on 2,000 queries), 3 random orderings for iid and type-blocks.

6. Results, in the order they were obtained

6.1 Small validation run (300 train / 100 val / 200 test)

Purpose: validate the whole chain before spending GPU hours (about 35 to 40 minutes). Mean F1 per evidence variant: narrow 0.52, wide 0.63, gold 0.67 with the base model; the 7B specialist adds roughly 0.06 to 0.11. So there is genuine headroom for deferral. Findings that shaped everything afterwards:

- PPO-Lagrangian initially appeared to win, but it was compared with human-only baselines. It re-retrieved on 54 to 57% of queries, and re-retrieval alone gives about +0.11 F1 at tiny cost, which a trivial "always re-retrieve" rule also gets. The comparison was not like-for-like.
- Training return (about 0.96) far exceeded test net reward (0.56 to 0.69): overfitting on 300 queries. Validation-based checkpoint selection was added.
- With only 200 test queries the CI half-width is about +/-0.05; no small difference could be resolved.
- Signals (AUROC for predicting a wrong answer): token entropy 0.713, supervised GBM 0.760, critic P(Yes) 0.62, cross-agent xgap/xent about 0.53 (chance: dropped from any claim).

6.2 Matched baselines and added signals (small run)

Adding wide_only / wide_spec / wide_gbm / wide_spec_gbm (same action set as the policy) removed the apparent advantage: differences to the matched baselines were within about +/-0.02 with all CIs spanning zero. Sampling-based semantic signals did not help (sem_agree 0.576, sem_ent 0.568 AUROC; overall GBM 0.760 to 0.766). Cross-model agreement was weak (0.612 AUROC after the specialist call). Argmax decoding left low budgets unused (human rate 0.000 at b=0.02 and 0.05) and produced a significant loss to wide_gbm at b=0.05 (-0.0485); sampling decoding spent the budget (0.010 and 0.043) and removed the loss. Conclusion at this stage: the signal ceiling and the near one-shot structure of the problem, not the policy, limit what can be gained.

6.3 Full run (3,000 / 1,000 / 2,000), per-query average-budget formulation



Figure 2. Net reward vs human budget on 2,000 test queries (PPO-Lagrangian, sampled decoding, 5-seed ensemble).

Method	b=0.02	b=0.05	b=0.10	b=0.20	b=0.30
wide_gbm	0.5552	0.5809	0.6165	0.6787	0.7343
wide_spec_gbm	0.5618	0.5880	0.6273	0.6850	0.7269

Method	b=0.02	b=0.05	b=0.10	b=0.20	b=0.30
PPO-Lagrangian (sampled)	0.5712	0.5954	0.6336	0.6899	0.7343
oracle (hindsight)	0.7296	0.7596	0.8096	0.9086	0.9649

Reference points without humans: auto_only 0.4594, always_specialist 0.4779, wide_only 0.5394, wide + specialist 0.5428.

- **PPO vs wide_gbm:** +0.011 to +0.017 net reward at b up to 0.10 with CIs excluding zero (lower bounds about +0.001 to +0.004); a tie at b=0.20 (+0.011) and b=0.30 (0.000).
- **PPO vs wide_spec_gbm:** +0.005 to +0.010, every CI spans zero. No significant win over the stronger heuristic at any budget. PPO's F1 is lower than wide_spec_gbm (0.606 vs 0.632 at b=0.02): it wins on net reward by calling the specialist on about 36 to 39% of queries instead of all.
- **Gain decomposition (Figure 3):** re-retrieval +0.080, specialist +0.019, both +0.083 over auto_only; humans add +0.016 (b=0.02) up to +0.195 (b=0.30) over wide_only.
- **Multiplicity:** about 30 CIs were computed; the b=0.05 lower bound (+0.0012) would not survive correction.

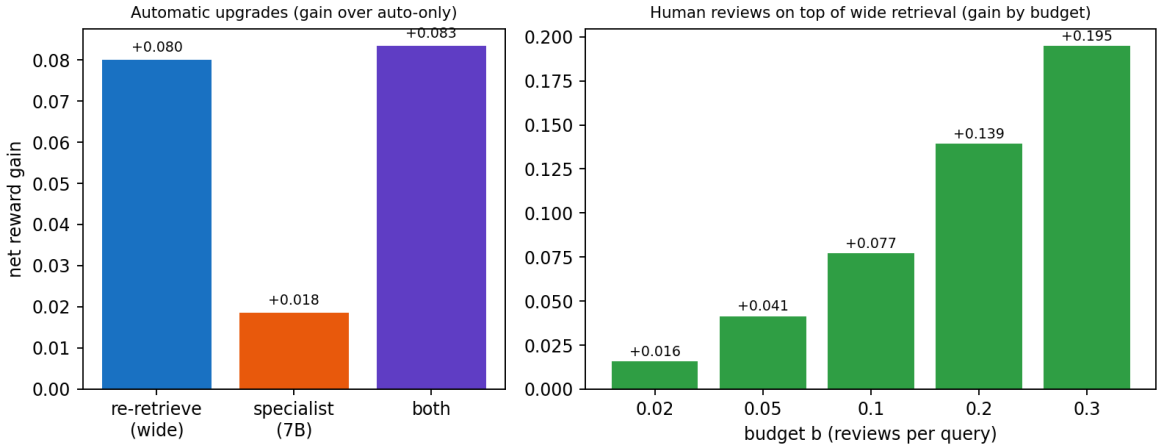


Figure 3. Where the reward comes from. Left: automatic upgrades. Right: human reviews on top of wide retrieval, by budget.

6.4 Signal audit and headroom

Signal / model	AUROC for predicting a wrong answer	Run
token entropy (best single signal)	about 0.68 (0.713 in the small run)	full
supervised GBM on all signals	about 0.77 (0.74 to 0.78 in the post-escalation cell)	full
cross-model agreement (xmodel_agree), post-escalation cell	0.652 (0.612 in the small run)	full
sampled-answer agreement / entropy	0.58 to 0.59	full
critic P(Yes)	about 0.62	small
critic-vs-NLI gap / entropy	about 0.53 (chance)	small

The oracle is 0.158 to 0.231 net reward above the learned policy at every budget. No policy can close that gap without better error-prediction signals (a stronger verifier, larger judge models, or calibrated ensembles).

6.5 Specialist-cost sensitivity (full size)

Because the specialist cost (0.05) is our assumption, we retrained and re-evaluated at other costs. At cost 0.0 and 0.02 always calling the specialist is nearly optimal: PPO was above wide_gbm by +0.015 to +0.052 but below wide_spec_gbm by 0.005 to 0.014 (several CIs excluding zero in the negative direction), using the specialist on only 58 to 74% of queries at cost 0.0. At cost 0.05 everything ties. The cost-0.10 block was lost to notebook output truncation and was not analysed. Reading: the policy roughly tracks the better of the two fixed rules, and a validation-chosen switch between them would likely match it. The ranking depends on the cost assumption.

6.6 Stream-level hard-budget policy (version 1.0-final)

Evaluation used windows of 1,000 queries under a hard cap, on three real-query streams and five budgets (15 cells), against the best of the static and paced heuristics. Budget violations: 0 windows over budget. Result: **0 wins, 12 ties, 3 losses**. The losses were at b=0.30 on the iid stream (-0.032), at b=0.30 on type-blocks (-0.035), and at b=0.20 on type-blocks (-0.015). At low budgets (up to 0.10) the policy matched the paced heuristic (differences within +/-0.01; at b=0.02 PPO was +0.006 to +0.011 ahead with CIs spanning zero). Budget pacing mattered less than expected: in the hard-first stress stream the paced heuristic beat the static one by only +0.005 to +0.012. Likely causes of the high-budget losses (not tested): the edge of the training budget range and weaker risk ranking than the supervised model.

6.7 Risk-stacked stream policy (version 1.1-stacked)

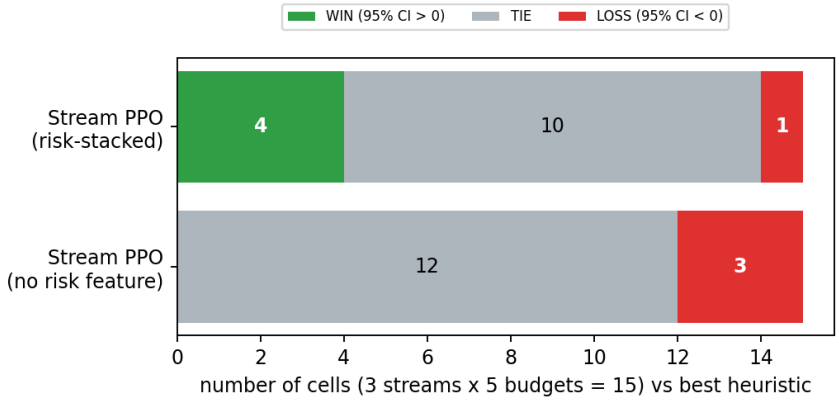


Figure 4. Verdicts against the best heuristic over 15 cells (3 streams x 5 budgets).

Cell (stream, budget)	PPO minus comparator (net reward)	95% CI	Verdict
iid, b=0.02	+0.0126	[+0.0007, +0.0247]	WIN
hard-first, b=0.02	+0.0170	[+0.0042, +0.0305]	WIN
hard-first, b=0.10	+0.0156	[+0.0017, +0.0299]	WIN
type-blocks, b=0.02	+0.0133	[+0.0018, +0.0248]	WIN
type-blocks, b=0.30	-0.0139	(excludes 0)	LOSS

- Stacking raised net reward over the plain stream policy in **all 15 cells**, by +0.005 to +0.021.
- In all 12 cells with b up to 0.20 the point estimate was positive (+0.001 to +0.017); at b=0.30 all three were negative (-0.004 to -0.014).
- Only 4 of 15 CIs exclude zero and their lower bounds are close to zero; with correction for 15 comparisons at most one or two would survive. Cells are not independent (same test queries and policies), so a sign test would be misleading.
- The likely mechanism (untested) is that the policy uses per-cell risk to decide when the specialist or re-retrieval is worth its cost, and uses the budget state to pace reviews.

6.8 Summary table of the study

Stage	Question	Outcome
Small run	Does the chain work; is there headroom?	Yes. Re-retrieval explains most early gains; overfitting and under-powered test set
Matched baselines	Does RL beat heuristics with the same actions?	No (ties); argmax decoding under-spent the budget
Full run, per-query CMDP	Same, at 2,000 test queries	Small gain over wide_gbm at b up to 0.10; tie with wide_spec_gbm; cost-dependent
Stream, hard cap (1.0-final)	Does budget-aware RL beat paced heuristics?	0 wins / 12 ties / 3 losses
Stream, risk-stacked (1.1-stacked)	Does a risk feature help the policy?	4 wins / 10 ties / 1 loss; +0.005 to +0.021 over unstacked in all cells
Confirmatory (1.2-confirm)	Does it replicate on fresh queries?	Prepared, not completed (Section 10)

7. Root-cause analysis: what we learned

- **The problem is nearly one-shot.** Each query involves two to four decisions and a terminal reward. When the best action is mostly a function of the final-state features, a supervised risk model plus fixed escalation is close to optimal, leaving RL little to add. RL should matter more in longer-horizon settings (critique-and-revise loops, drifting streams with a shared budget).
- **Information, not optimisation, limits the result.** AUROC around 0.68 to 0.77 caps what any deferral rule can achieve; the oracle gap (0.16 to 0.23) is an order of magnitude larger than any policy difference we measured.
- **Most reward comes from cheap automatic actions.** Wide re-retrieval (+0.080) dwarfs everything a policy adds on top, which is why early comparisons against human-only baselines were misleading.
- **Cost assumptions drive rankings.** The value of the specialist is almost exactly its assumed cost at 0.05; changing it flips which fixed rule is best.
- **Evaluation details matter as much as algorithms.** Argmax decoding of a budget-feasible policy silently left the budget unused; matched-action-set baselines changed the conclusion; the per-query average constraint was a relaxation of the "per 1,000 queries" requirement.
- **Stacking works because it separates concerns:** supervised learning ranks risk well; RL decides pacing and escalation. The learned policy is best described as a decision layer on top of a calibrated risk model, not a replacement for it.

8. What can and cannot be claimed

Supportable now	Not supportable
A real-execution, no-synthetic-data replay environment for budgeted human-in-the-loop orchestration with ground truth at every step, a hard budget per 1,000 queries (0 violations) and a full audit trail.	"RL beats strong heuristics." Against a plain policy the result is parity; the stacked policy shows small, individually borderline gains that are not yet independently confirmed.
A decomposition of where reward comes from (re-retrieval, specialist, human reviews) and an AUROC audit of the signals.	"Production-ready" or "bridges to production". All results are offline replay; live agent serving and live Kafka streaming were not validated.
The finding that supervised risk + fixed escalation + budget pacing matches a plain RL policy, and that stacking the risk score into the RL state gives a small, consistent improvement at budgets up to 0.20.	Generalisation to legal or enterprise documents, other model families, or real human reviewers. Only HotpotQA, Qwen 3B/7B and an oracle human were tested.

Suggested wording for a paper: "Using risk-stacked, budget-aware policies we observe a small but consistent improvement over paced supervised heuristics at low-to-moderate review budgets (b up to 0.20) on HotpotQA, with a hard cap of K reviews per 1,000 queries enforced by construction. The improvement is modest in absolute terms and

was confirmed / not confirmed on a held-out fresh split (Section 10)."

9. Limitations and threats to validity

- **Researcher degrees of freedom.** Decoding, baselines and the stacking idea were chosen after looking at test results. The reported improvement may be optimistic; this is why the fresh-split confirmation exists.
- **Multiple comparisons.** 15 CIs per policy variant, no correction in the main report.
- **Single dataset and models.** HotpotQA (Wikipedia multi-hop QA), Qwen2.5-3B/7B only; no legal or enterprise-domain test.
- **Oracle human.** The reviewer always supplies the gold answer. A `human_acc` switch exists but was not swept in the main results.
- **Assumed costs.** Re-retrieve 0.02 and specialist 0.05 are our choices; rankings change with them.
- **Seed variance.** Policies are 3-seed ensembles; per-seed variance is reported only in the confirmatory protocol.
- **Replay, not live.** No live-agent serving; vLLM cache rebuilds differ slightly (about 0.002 to 0.005) between runs.
- **Stress stream.** Hard-first ordering uses ground-truth difficulty and is not a deployment estimate.

10. Confirmatory evaluation on fresh queries

Design. 2,000 new questions from the official dev split (shuffled positions 2000 to 4000; the main test used 0 to 2000), disjoint from all earlier queries (checked by id). Everything is frozen: the three stacked policies, the risk models (refit deterministically from the main training set and verified against the saved training feature), PCA/z-score transforms, validation-derived thresholds, sampling decoding and the comparator.

Pre-registered primary hypotheses (3). On the iid stream (W=1,000, 3 orderings) at b = 0.02, 0.05, 0.10: mean per-query net reward of the stacked policy is greater than that of paced_wide_spec_gbm. One-sided paired bootstrap (10,000 resamples), Holm correction over the 3 tests, alpha = 0.05. Everything else (b = 0.20 and 0.30, other streams, per-seed results, the no-risk ablation) is exploratory. The evaluation refuses to run if the code hash or any frozen-artefact hash differs from the registration.

Status. The pre-registration was generated on 2026-10-18 at 14:52:51 UTC (code hash b8dd4ab4..., main cache 3bd8320d..., policies d465d18c..., 1a279fbe..., 85bd3fbe...). The fresh cache was built successfully (mean F1 per cell, evidence x answerer: narrow 0.478 / 0.543, wide 0.561 / 0.616, gold 0.633 / 0.724 for base / specialist, consistent with the main cache). The evaluation step then failed because the registration file (prereg.json) was not found in the working directory, most likely lost with a session change (the exact cause was not established); the registration is to be regenerated and its hashes compared with the printed ones before evaluating. **No policy has been evaluated on the fresh queries yet, so no confirmatory result exists.**

How to read the eventual outcome. 2 or 3 of 3 hypotheses confirmed: the small improvement replicates. 0 or 1 confirmed: the main-test gains were largely noise or selection effects and should be reported that way; the contribution then rests on the environment, the signal audit and the parity finding. Either outcome is reportable.

11. Reproducibility and artefacts

Artefact	Purpose
hitl-orchestrator.zip (src/hitl/*.py, tests, README)	Complete code: pipeline stages, replay environment, PPO-Lagrangian, stream trainer, baselines, evaluation, audit, confirmatory protocol
hitl_kaggle_notebook.ipynb	Original all-in-Kaggle notebook (cache build + per-query training/evaluation)
hitl_FINAL.ipynb	CPU-only final run: stream policy (optionally risk-stacked) on the existing cache; generates FINAL_REPORT.md
hitl_CONFIRM.ipynb	Pre-registration, fresh cache build (GPU), frozen confirmatory evaluation
hitl_run_outputs.zip, hitl_final_outputs.zip, hitl_final_stacked_outputs.zip	Main cache, unstacked policies and results, stacked policies, risk feature and results. Keep them: the confirmatory run needs the stacked policies and the main cache

Version history

Version	Change
0.1 to 0.5	Pipeline stages, replay environment, PPO-Lagrangian; matched baselines; validation checkpoint selection; sampled-answer signals; cross-model agreement
0.6-ppo-decode	Sampled and thresholded PPO decodings (fix for unused budgets under argmax)
0.7-comparisons-csv	All CIs saved to a CSV; quieter cost sweep
1.0-final	Stream-level hard budget, drift streams, paced baselines, auto-generated report, one-command run
1.1-stacked	Out-of-fold supervised risk score as a policy input; better coverage of high training budgets
1.2-confirm	Fresh-split support, frozen-artefact pre-registration, Holm-corrected confirmatory evaluation

Appendix A. Engineering problems met and how they were fixed

Problem	Cause	Fix
torchaudio CUDA mismatch crashed sentence-transformers	pip install of vLLM brought a newer PyTorch (CUDA 13) while Kaggle's torchaudio was built for CUDA 12.8	Uninstall torchaudio (unused). Do not remove torchvision: vLLM requires it at startup
vLLM engine failed at startup ("No module named torchvision")	torchvision had been removed as a precaution	Reinstall torchvision without touching other packages
Project not found / old code loaded	Kaggle auto-extracts uploaded zips and pins dataset versions	Cell that scans all attached inputs, reads each copy's version tag and uses the one that matches; version guard
meta.json / cache not found	Stage cells not run in the current session, or cache wiped	Run stages in order; resumable stores; size guard against mixed sizes
Notebook outputs too long to read	Per-iteration training logs	Logs redirected to files; only summaries printed; comparisons.csv
Policy never chose HUMAN at low budgets	Argmax decoding of a stochastic budget-feasible policy	Sampling decoding; threshold decoding as diagnostic
Unfair first comparison	Baselines lacked re-retrieve / specialist actions	Matched-action-set baselines (wide_gbm, wide_spec_gbm, paced variants)
Overfitting on 300 queries	Policy could memorise question embeddings	Validation-based checkpoint selection; full-size training data
Confirmatory evaluation could not find prereg.json	Working directory changed between registration and evaluation	Re-run registration, compare printed hashes; keep outputs with persistence on

Appendix B. Main hyperparameters

Item	Value
PPO network	2 hidden layers of 128 tanh units; separate reward and cost critics (cost critic unused in stream mode); Adam, lr 3e-4; clip 0.2; entropy coefficient 0.01
Per-query PPO-Lagrangian	300 iterations x 1,024 episodes; 6 epochs; minibatch 512; GAE lambda 0.95; dual lr 1.0, lambda init 0.5, max 20; 5 seeds per budget
Stream PPO	250 iterations (200 in 1.0-final); 32 windows of W=200 per iteration; 4 epochs; minibatch 2,048; GAE lambda 0.98; budgets sampled in [0.02, 0.30] (log-uniform in 1.0, half uniform from 1.1); 3 seeds; reward scaled by 10 / W
Costs	re-retrieve 0.02; specialist 0.05; human review = constraint, F1 = 1 (oracle)
Evaluation	W=1,000; orderings: 3 (iid, type-blocks), 1 (hard-first); bootstrap 2,000 (main) / 10,000 (confirmatory) resamples
Sampling signals	5 samples per cell, temperature 0.7, max 24 tokens
Risk models	HistGradientBoosting, depth 4, lr 0.05, 200 iterations; one per evidence/answerer cell; 5-fold out-of-fold for training queries

Note on numbers: every figure in this report is taken from the notebook outputs and run reports produced during the project (FINAL_REPORT.md files, evaluation logs). Exact per-cell tables are in those files; this document reports the values needed for the argument.